Software Requirements Specification

Voice Assistant for Multimedia Controls

Team 3 | Version 1.3 | 11 November 2026

Prepared for Dhanush S S and Team 3. Revised prototype baseline based on VoiceAssistant_Project_Complete_Guide.txt. Instructor approval of scope changes is pending. Requirement statements specify expected behavior; they do not certify implementation or test acceptance.

1 Introduction

1.1 Purpose

This SRS defines the ESP32 and Windows laptop prototype, its interfaces, supported voice commands, responsibilities and acceptance conditions. It supports implementation, cross-testing and academic review. It supersedes the earlier on-board tinyML and BLE design only for this proposed prototype baseline; the changes in Section 8 require review before replacing submitted course documents.

1.2 Scope

An INMP441 microphone supplies audio through I2S to an ESP32. On request from Python, the ESP32 streams five seconds of PCM audio over USB serial. The laptop saves a WAV file, transcribes it locally with Faster-Whisper, classifies a keyword, maps an action and uses PyAutoGUI to send a Windows media key. Each invocation processes one recording. The objective remains voice control of multimedia playback.

Cloud transcription is excluded. A cached model permits local recognition without internet; initial model installation may require a download. Internet needed by a streaming player is separate from recognition. The laptop is an essential processing component.

1.3 Definitions

PCM means pulse-code modulation; WAV is the audio container. I2S is the microphone digital audio bus. USB serial is the ESP32-to-laptop transport. An intent is a classified command; an action is the mapped execution identifier. RTM means requirements traceability matrix. BLE HID means Bluetooth Low Energy Human Interface Device and is outside this prototype baseline. A media toggle changes the current state rather than selecting a guaranteed state.

1.4 References and precedence

The supplied complete project guide is the reference for this revision. Prior SRS version 1.2, SAD, STP and backlog remain historical baselines requiring coordinated reconciliation. The guide is descriptive input, not proof of test success. The 51-second prototype demo and 7 November test reports provide limited evidence and do not automatically establish this SRS's acceptance criteria.

2 Team and assigned responsibilities

Member and SRN	Assigned responsibility	Independent cross-tester
Dhanush S S — PES2UG24CS154	Project Manager and Integration Lead. Own main.py, run configuration, COM-port setup, end-to-end integration, repository coordination, SRS assembly and release evidence.	G S Dheekshith
G S Dheekshith — PES2UG24CS166	Audio Module Lead. Own VoiceAssistant_ESP32.ino and receive_audio.py, INMP441 wiring, I2S conversion, bounded streaming, serial framing and WAV creation.	Harsh Mannan
Harsh Mannan — PES2UG24CS183	Recognition Module Lead. Own speech_to_text.py and command_classifier.py, local model configuration, normalization, supported keywords and recognition evaluation. Coordinate SAD/STP reconciliation with all owners.	Chiranthan Shankar
Chiranthan Shankar — PES2UG24CS137	Output and Maintenance Lead. Own command_mapper.py and media_controller.py, action mapping, PyAutoGUI dispatch, execution outcomes and media behavior documentation.	Dhanush S S

These are responsibility assignments, not claims of historical authorship. Each owner raises their module PR; Dhanush coordinates review and merge. A cross-tester records actual results independently. All four jointly validate the complete pipeline and contribute their module documentation. Work in another owner's module requires coordination.

3 Overall description

3.1 Operating environment

ESP32-WROOM development module with an INMP441, USB data cable, Windows laptop, available serial port, ESP32 Arduino core and Python virtual environment. Python dependencies are pyserial, faster-whisper and PyAutoGUI. wave, struct, pathlib and time are standard-library modules. Pin and dependency configuration must match the actual build; exact tested software versions are recorded with test evidence.

3.2 User workflow

Upload the firmware using Arduino IDE after selecting the actual board and port. Close Serial Monitor before Python opens the port. Run python main.py COM3, replacing COM3 with the real port. Wait for the recording prompt, speak during the five-second window and observe the recognized text, intent, mapped action and outcome. Run the program again for another recording.

3.3 Data flow

INMP441 -> I2S -> ESP32 -> USB serial -> receive_audio.py -> recording.wav -> speech_to_text.py -> transcript -> command_classifier.py -> intent -> command_mapper.py -> action -> media_controller.py -> Windows media key -> active media environment.

main.py coordinates these stages through public function interfaces. A successful key dispatch and a visibly successful media action are distinct observations.

3.4 Constraints

The ESP32 performs capture and transport, not speech recognition. Firmware streams small chunks rather than storing 160,000 bytes in a global recording buffer. Laptop inference speed depends on CPU and model availability. The fixed capture window alone takes five seconds, so the previous three-second end-to-end limit cannot be retained as a prototype acceptance target. Keywords do not understand general conversation or negation. PLAY and PAUSE use the same toggle key and cannot guarantee the requested playback state.

4 External interfaces

4.1 Hardware interface

INMP441 signal	ESP32 connection
VDD	3V3
GND	GND
SCK or BCLK	GPIO 26
WS or LRCL	GPIO 25
SD	GPIO 33
L/R	GND for left channel

This wiring comes from the supplied guide. Confirm the board pinout and connections before powering. Audio is mono at 16,000 samples per second, signed PCM16 output; I2S input slots are 32 bits. The conversion must preserve signed sample interpretation.

4.2 Serial protocol

Field	Contract
Link	USB serial at 921600 baud
Request	Python sends ASCII r
Header	Four ASCII bytes AUD0 followed by an unsigned 32-bit little-endian payload length
Audio	160,000 bytes for 16,000 Hz x 5 seconds x 2 bytes x 1 channel
Trailer	Four ASCII bytes END0

The framed response contains 160,012 bytes excluding the request. No diagnostic text may be interleaved with the binary packet. The receiver must handle short reads until the required byte count is collected. Port opening may reset the board; readiness delay, finite read timeout and recovery behavior must be documented and tested rather than assigned invented timing values.

4.3 Software contracts

Module interface	Input and output
record_from_esp32(port)	COM-port identifier to validated WAV path, or reported capture failure
transcribe_audio(audio_file)	WAV path to joined transcript string, or reported model/transcription failure

Module interface	Input and output
classify_command(transcript)	Text to supported intent or UNKNOWN
map_command(intent)	Intent to action identifier or unknown
execute_action(action)	Supported action to Windows key dispatch and reported outcome; invalid actions cause no key dispatch

Function signatures and error representation must be reconciled with the code before formal API acceptance. These contracts describe observable behavior without prescribing an unverified exception or return-type convention.

4.4 Commands and execution semantics

Spoken keyword	Intent	Action identifier	PyAutoGUI key
play	PLAY	media_play	playpause
stop	PAUSE	media_pause	playpause
next	NEXT	media_next_track	nexttrack
previous	PREVIOUS	media_previous_track	prevtrack
mute	MUTE	audio_mute	volumemute
increase	VOLUME_UP	volume_increase	volumeup
decrease	VOLUME_DOWN	volume_decrease	volumedown

Stop is an alias for the PAUSE intent, not a guaranteed media-stop operation. Mute toggles host sound; it does not disable the microphone. Volume actions may issue multiple presses; the exact current press count must be documented from code before testing. Resume is not a separate supported keyword in the guide. Multiple matched keywords and negation require an approved policy; do not claim robust ambiguity rejection from simple matching.

5 Functional requirements and traceability

Existing IDs are preserved. Reassigned meanings below are explicit proposed revisions, not silent replacements of the prior baseline. Acceptance case IDs remain references for STP revision; their old procedures do not automatically apply.

Requirement and owner	Revised requirement	Acceptance and test reference
VAMC-F-001 — Dheekshith	Wake-word monitoring is deferred; recording is initiated by a serial request.	Historical TC-WW-01 is deferred, not passed. Verify request-triggered behavior under F-002.
VAMC-F-002 — Dheekshith	Firmware shall capture five seconds of mono 16 kHz PCM16 audio per valid request and stream bounded chunks.	TC-WW-02 revised: request recording, validate 160,000 payload bytes and sample format; inspect bounded buffers. Actual execution record required.
VAMC-F-003 — Dheekshith	Physical microphone mute remains deferred.	TC-WW-03 deferred. Host MUTE does not satisfy this requirement.
VAMC-F-004 — Harsh	The laptop shall transcribe the captured WAV locally using Faster-Whisper base, CPU, int8 and beam size 1.	TC-ASR-01 revised: cached model transcribes known recorded input with recognition-network access unavailable; record actual transcript and inference time.

Requirement and owner	Revised requirement	Acceptance and test reference
VAMC-F-005 — Harsh	The classifier shall lowercase text, remove punctuation, split into words and identify the seven documented keywords.	TC-ASR-02 revised: exercise all keywords, punctuation and case variants; compare with the command table.
VAMC-F-006 — Harsh	No supported keyword shall yield UNKNOWN, and unknown actions shall not dispatch a media key.	TC-ASR-03 revised: empty, unrelated and unsupported text cause no key. Multiple-keyword and negation behavior is a documented limitation pending a policy.
VAMC-F-007 — Chiranthan	The mapper shall implement exactly the seven intent-to-action mappings in Section 4.4.	TC-CMD-01 revised: verify all mappings and unknown intent behavior. Resume-to-Mute vocabulary change requires approval.
VAMC-F-008 — Chiranthan	The controller shall dispatch the documented Windows key for each valid action using PyAutoGUI.	TC-CMD-02 revised: observe all seven dispatches and host behavior; explicitly record toggle limitations and platform failures. Prior BLE transport and two-second criterion are deferred.
VAMC-F-009 — Chiranthan	BLE retries and radio backoff remain deferred.	TC-CMD-03 deferred. Serial recovery is governed by F-012, not counted as BLE retry acceptance.
VAMC-F-010 — Chiranthan	Console output shall expose the transcript, detected intent, mapped action and execution outcome.	TC-CMD-04 revised: compare console stages with actual processing; do not report successful execution after a dispatch failure. LED/buzzer timing is deferred.
VAMC-F-011 — Dhanush	The application shall accept a user-supplied COM-port argument and display usage when it is missing.	TC-CONN-01 revised: correct-port, missing-argument, unavailable-port and occupied-port runs produce appropriate outcomes. Wi-Fi provisioning is deferred.
VAMC-F-012 — Dheekshith with Dhanush integration	Receiver shall validate AUD0, length, complete payload and END0 before producing a valid recording.	TC-CONN-02 revised: split reads, bad markers, unexpected length, truncation and unavailable port fail without dispatching media keys; subsequent clean run is possible. BLE pairing is deferred.
VAMC-F-013 — Dhanush	Physical factory reset remains outside this prototype baseline.	TC-CONN-03 deferred; no simulated reset is accepted as device credential erasure.
VAMC-F-014 — Chiranthan	Diagnostic reporting shall describe processing and failures without disclosing secrets.	TC-DIAG-01 revised: normal and injected-error runs provide stage information; circular firmware logs and retention policy are deferred.
VAMC-F-015 — Chiranthan	OTA firmware update remains deferred; firmware is uploaded through Arduino IDE.	TC-DIAG-02 deferred. A USB upload is not OTA acceptance.

5.1 Error handling expectations

Serial framing or length errors, incomplete audio, model-load/transcription failures, UNKNOWN intent and unsupported actions shall stop the current run before an unintended key is dispatched. Resources must be released on exit. Current robust error handling is identified as unfinished in the guide; these are acceptance expectations, not statements that every path is implemented. Finite timeouts, serial resynchronization and ambiguous-input policy require agreement and tests.

6 Nonfunctional and security requirements

Existing ID	Prototype disposition and verification
VAMC-NF-001	Original p90 <= 3 seconds over 50 wake-to-action trials is deferred because fixed capture takes 5 seconds and no wake engine exists. Measure request-to-capture-end, transcription and dispatch separately; no replacement pass threshold is invented. Owner Dheekshith; joint integration by Dhanush.
VAMC-NF-002	Original >= 90 percent over 100 utterances at SNR >= 15 dB remains a recognition evaluation target, not a measured result. Harsh owns the seven-keyword dataset and exact-match intent scoring; Chiranthan cross-tests. No reported accuracy is inferred from the short demo.
VAMC-NF-003	Original average setup <= 5 minutes for 5 first-time users requires review because Windows dependencies and model preparation changed setup. Harsh coordinates and Dheekshith cross-tests; separate environment installation from ready-environment run time in measurements.
VAMC-NF-004	Original >= 8-hour idle listening on a 2000 mAh battery is deferred; this prototype is USB/laptop dependent and does not continuously listen. Dheekshith retains ownership.
VAMC-NF-005	Modular replacement remains required. Replace one module with a compatible test double without editing other modules; record regression results. Dheekshith coordinates and Dhanush independently verifies.
VAMC-SR-001	Local speech processing remains required. With cached model and recognition network disabled, capture and classify; inspect network behavior for absence of externally transmitted audio. Harsh owns; Chiranthan cross-tests. Initial model download is explicitly separate.
VAMC-SR-002	Authenticated BLE pairing is deferred because serial is used. Dhanush retains ownership of future pairing; TC-SEC-02 is not accepted by this prototype.
VAMC-SR-003	Encrypted Wi-Fi storage is deferred because this baseline has no Wi-Fi provisioning. Dhanush retains ownership; this does not erase or certify previously developed credential code.
VAMC-SR-004	Original no-retained-raw-audio requirement is not satisfied merely by local processing. The guide writes recordings/recording.wav. Dheekshith owns an explicit retention/deletion decision and error-path cleanup tests; Harsh cross-tests. Until agreed and verified, TC-SEC-04 remains unresolved.
VAMC-SR-005	OTA signatures remain deferred with OTA. Chiranthan owns future implementation and Dhanush cross-tests; TC-SEC-05 remains unaccepted by this prototype.

Audio WAV files contain user speech and shall not be committed to the public repository as test evidence without permission. Recordings, model caches, virtual environments and secrets should be excluded from version control. A local transcript printed for the prototype differs from the earlier command-ID-only logging rule and must be documented in the SAD/STP revision.

7 Validation and acceptance plan

7.1 Required test groups

Audio tests verify wiring/channel configuration, signed conversion, frame size and full serial packet reconstruction. Protocol tests include partial reads, malformed headers/trailers, wrong lengths and truncation. Recognition tests verify cached-model local inference and the approved vocabulary using actual audio. Classifier/mapper unit tests include punctuation, case, empty and unknown inputs. Windows execution tests verify each mapped key and observed host behavior, including playpause and mute toggles. Integration tests cover a valid full run and failures before dispatch.

7.2 Evidence and status

Each test record identifies requirement ID, revised STP procedure, source commit, device and software versions, input, expected and actual result, date, tester and artifacts. N means not started, P in progress, T ready for test, A accepted, F failed and B blocked. Only an independent tester records A or F. A deferred requirement is a scope disposition, not an Accepted result.

The existing 7 November reports measured 93 C++ cases and 10 Python methods; they are baseline-specific evidence. They do not prove all guide-described code paths, real serial transport or Faster-Whisper inference. The supplied prototype video provides limited visual demonstration. Project-lead issue closures reflect reported completion and are not independent measurements. No new acceptance status is set by this SRS.

7.3 Exit criteria

All active prototype requirements have traceable actual results; error paths prevent unintended dispatch; each of the seven actions is demonstrated on the chosen Windows environment; local inference is tested with model cached; toggle limitations and retention behavior are disclosed; outstanding defects and deferred scope are approved; independent reviewers sign the relevant records. Course acceptance of the changed baseline remains an instructor decision.

8 Change and deviation register

Change from previous SRS	Guide-aligned revision	Approval consequence
ESP32-WROOM tinyML recognition	Faster-Whisper on Windows CPU	Laptop becomes mandatory; update F-004, SAD and STP.
BLE HID to paired host	USB serial input and PyAutoGUI host output	Revise F-008/F-012; defer pairing and BLE security/retry requirements.
Play Pause Resume Next Previous Volume Up Volume Down	Play Stop Next Previous Mute Increase Decrease	Resume removed from current vocabulary; Mute added; Stop maps to PAUSE. Explicit instructor scope review needed.
Continuous wake word	One requested five-second recording per run	Defer F-001/F-003 and continuous/battery assumptions.
Guaranteed discrete playback action	Shared playpause toggle	Disclose state uncertainty; no guaranteed PLAY or PAUSE state.
No retained raw speech	WAV file written to disk	SR-004 remains unresolved pending retention policy and evidence.
Setup and maintenance on embedded device	CLI port selection and manual USB upload	Wi-Fi portal, reset, LED/buzzer, OTA and signing deferred.
Three-second end-to-end target	Five-second capture plus processing	Preserve original target historically; replacement threshold pending approval.

Original source files, completed portable modules and their tests must be preserved. The revised baseline must be reconciled into SAD, STP, backlog and repository descriptions through a dedicated documentation change after approval. Existing requirement IDs and TC references are retained here so the scope changes remain auditable.

9 Assumptions and future scope

The user has a supported Windows media environment and permission to dispatch media keys. Only one process owns the COM port at a time. The cached Faster-Whisper base model is available; its initial download and installation are planned separately. Actual sample conversion and driver compatibility require testing on the submitted firmware.

Wake word, continuous listening, voice-activity detection, on-board inference, BLE HID, state-aware playback, richer language understanding, application launch/close, GUI and additional commands are future scope. They are not claimed as implemented. No internet speech API is required by the recognition path.

10 Revision and approvals

Version	Date	Description
1.2	Prior baseline	ESP32-WROOM tinyML and BLE design; retained for change comparison.
1.3	11 November 2026	Guide-aligned ESP32 capture and Windows local-recognition prototype; all four members assigned; changes and deferred criteria recorded.

Prepared for Team 3 under Dhanush S S's requested SRS revision. All members review their assigned sections. Team lead approval, independent test acceptance and instructor approval must be recorded by the responsible people; no signature or approval date is prefilled.